HSE711 Final Project

Greg Fisher

2026-10-18

1) Download the Sample RNA-seq Count Matrix and associated Metadata.

```
# Read in both data sets
count.df <- read.csv(file = "data/counts.csv", header = TRUE)
meta_data.df <- read.csv(file = "data/meta_data.csv", header = TRUE)
```

Data wrangling over the next sequence of steps

```
# Use only 5 columns of data in the meta_data dataframe to make processing faster
metadata.modified <- meta_data.df %>%
  dplyr::select(1,10,23,29,41,86) %>%
  # The barcode variable will eventually be used to join the metadata to the count data
  # The hyphen in the barcode variable needs to be changed to a dot
  mutate(barcode = str_replace_all(barcode, "-", "."))
```

```
# Convert count.df to long format
# This is required for many of the ggplot graphs and many other functions in R
# Use the pivot_longer function to convert the count dataframe to one I am calling
# dat.long which will end up containing three columns - the Ensembl ID, one called
# Samples which will contain all of the individual sample values and one called
# Counts which will contain the gene_expression count values

dat.long <- count.df %>%
  pivot_longer(
    cols = -X,         # Pivot all columns except the one containing the Ensemble ID
    names_to = "Samples",     # Column name for your old wide headers
    values_to = "Counts"     # Column name for your data values
  )
```

```
# This code block is where I join the dat.long dataframe for the counts to the
# metadata dataframe for the additional information needed for the assignment
# The results of the join are stored in a new dataframe.
# dat.long$Samples is joined on metadata.modified$barcode
dat.long.enriched <- dat.long %>%
  mutate(X = sub("\\.[^.]*$", "", X)) %>% # Strip off version number
  inner_join(., metadata.modified, by = c("Samples" = "barcode"))
```

```
# This code block is used to map the Ensembl IDs to their actual gene symbol.

# Load required libraries
library(AnnotationDbi) # provides an R interface to query SQLite-based biological
                       # annotation database
library(org.Hs.eg.db)  # Used to access genome-wide human annotations based primarily on
```

```
                        # Entrez Gene identifiers

# The Ensemble ID has a "version" number on it that needs to be stripped off because
# the gene_symbols generated do not have it.
count.df.mod <- count.df %>%
  mutate(X = sub("\\.[^.]*$", "", X))

# Now need to pull the processed Ensemble ID's out into a vector
ensembl.ids <- count.df.mod %>% pull(X)

# This is the heart of the process where the Ensemble ID is mapped to Gene Symbols
# Uses he mapIds() function from AnnotationDbi. Maps identifiers for genomic data by
# connecting a set of input keys to target columns using annotation databases
gene_symbols <- mapIds(
  x = org.Hs.eg.db,
  keys = ensembl.ids,
  keytype = "ENSEMBL",
  column = "SYMBOL",
  multiVals = "first" # Keeps the first match if duplicate mappings occur
)

# Store the results in a dataframe "df" for use in a following join operation
df <- data.frame(names(gene_symbols),gene_symbols)
```

```
# This is where I merge the previously created dataframe "df" that contains the
# actual mapping from Ensembl ID to gene using an inner join
dat.long.genes <- dat.long.enriched %>%
  inner_join(., df, by = c("X" = "names.gene_symbols."))
```

All of the data wrangling has been completed up to this point. Additional steps will be required when we create the heat map

2) Gene selection and Summary Statistics

```
# I am choosing the gene BRCA1 to study
# BRCA1 is a human tumor suppressor genes that normally produce proteins to repair damaged
# DNA and prevent uncontrolled cell growth

get_mode <- function(x) {
  uniq_x <- unique(x)
  uniq_x[which.max(tabulate(match(x, uniq_x)))]
}

# Using dplyr filter out only BRCA1 and do summary statistics
summary_stats <- dat.long.genes %>%
  filter(gene_symbols == 'BRCA1') %>%
  summarize(
    mean = mean(Counts, na.rm = TRUE),
    min = min(Counts, na.rm = TRUE),
    max = max(Counts, na.rm = TRUE),
    median = median(Counts, na.rm = TRUE),
    mode = get_mode(Counts),
    sd = sd(Counts, na.rm = TRUE)
  )

summary_stats
```

```
## # A tibble: 1 x 6
##    mean   min   max median  mode    sd
##   <dbl> <int> <int>  <int> <int> <dbl>
## 1 1304.    21  8050    919   429 1137.
```

```
# BRCA2 Summary Statistics

get_mode <- function(x) {
  uniq_x <- unique(x)
  uniq_x[which.max(tabulate(match(x, uniq_x)))]
}

# Using dplyr filter out only BRCA1 and do summary statistics
summary_stats <- dat.long.genes %>%
  filter(gene_symbols == 'BRCA2') %>%
  summarize(
    mean = mean(Counts, na.rm = TRUE),
    min = min(Counts, na.rm = TRUE),
    max = max(Counts, na.rm = TRUE),
    median = median(Counts, na.rm = TRUE),
    mode = get_mode(Counts),
    sd = sd(Counts, na.rm = TRUE)
  )

summary_stats
```

```
## # A tibble: 1 x 6
##    mean   min   max median  mode    sd
##   <dbl> <int> <int>  <int> <int> <dbl>
## 1  749.    17  7206    529   356  717.
```

```r
# ATM Summary Statistics

get_mode <- function(x) {
  uniq_x <- unique(x)
  uniq_x[which.max(tabulate(match(x, uniq_x)))]
}

# Using dplyr filter out only BRCA1 and do summary statistics
summary_stats <- dat.long.genes %>%
  filter(gene_symbols == 'ATM') %>%
  summarize(
    mean = mean(Counts, na.rm = TRUE),
    min = min(Counts, na.rm = TRUE),
    max = max(Counts, na.rm = TRUE),
    median = median(Counts, na.rm = TRUE),
    mode = get_mode(Counts),
    sd = sd(Counts, na.rm = TRUE)
  )

summary_stats
```

```
## # A tibble: 1 x 6
##    mean   min   max median  mode    sd
##   <dbl> <int> <int>  <int> <int> <dbl>
## 1 3351.   233 23455   2817  1909 2315.
```

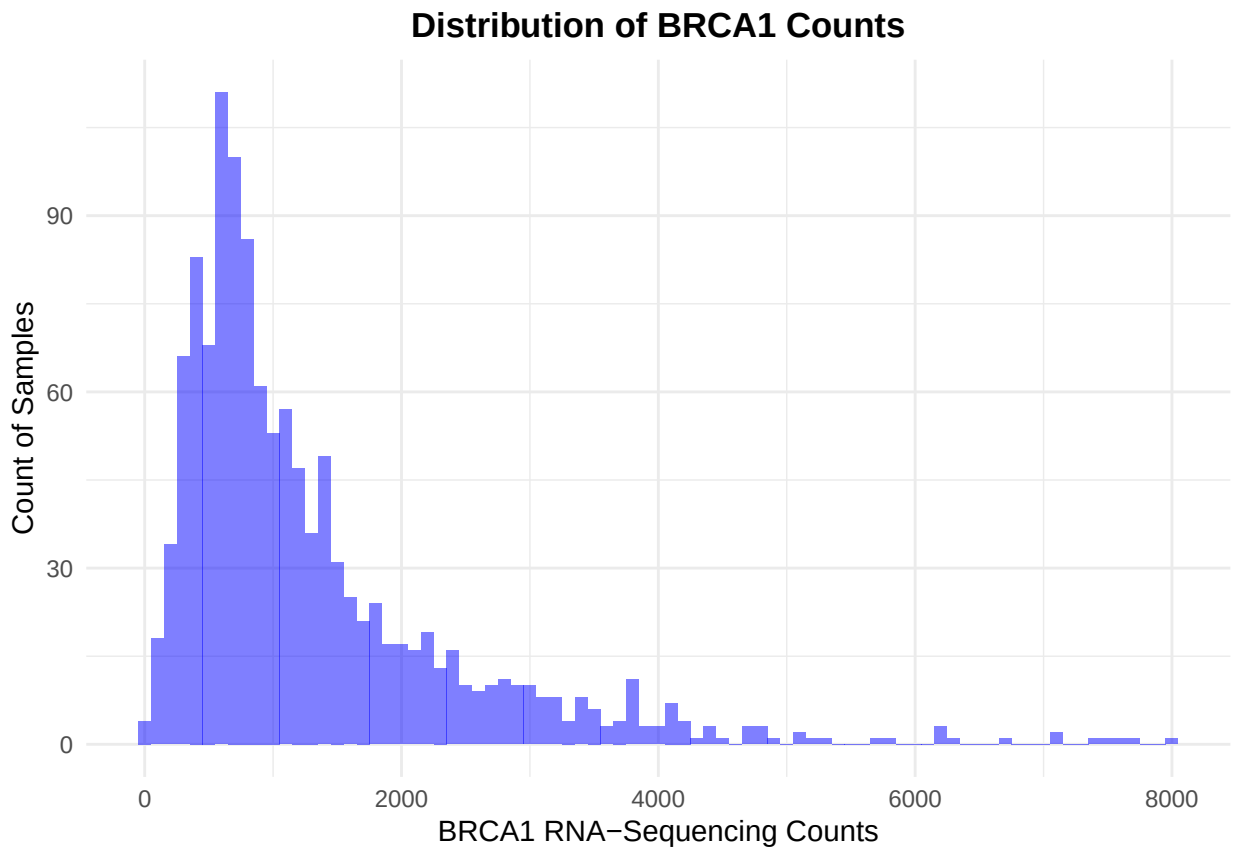
3) Visualization

```
# a) Create a Histogram: Use ggplot2 to generate a histogram of the count data for the selected gene.
# This visualization should effectively display the distribution of the counts.

# Using dplyr, filter for the BRCA1 gene and use the ajcc_pathologic_stage as fill aesthetic.
# The ajcc_pathologic_stage is a standardized cancer classification assigned after a
# patient undergoes surgical resection of the primary tumor.
# Generate a histogram using geom_histogram from ggplot2()

#png("images/Distribution of BRCA1 Counts.png",
#        width = 800, height = 600, res = 100)

dat.long.genes %>%
  filter(gene_symbols == 'BRCA1') %>%
  ggplot(., aes(x=Counts)) +
    geom_histogram(alpha = 0.5,
                   fill = "blue",
                   binwidth = 100,
                   na.rm = TRUE) +
     labs(
        title = "Distribution of BRCA1 Counts",
        x = "BRCA1 RNA-Sequencing Counts",
        y = "Count of Samples"
     ) +
     theme_minimal() +
     theme(
       plot.title = element_text(
         face = "bold",
         hjust = 0.5
       )
     )
```


Distribution of BRCA1 Counts
Count of Samples
0
30
60
90
0
2000
4000
6000
8000
BRCA1 RNA−Sequencing Counts

```
#dev.off()
```

```r
# b) Create a Scatter Plot: Select a second gene from the dataset. Create a scatter plot using
# ggplot2 to compare the count data of the two selected genes.

# make a vector of the genes of interest - BRCA2 is also a human tumor suppressor gene
genes.of.interest <- c('BRCA1', 'BRCA2')

# Using dplyr, filter for the genes of interest. Then, using the select verb, select all columns
# except column 1 prior to pivoting back from the current long form of data to the wide form
# This is required for the scatter plot.
# Use the sample_type as the factor variable in the plot

png("images/BRCA1 vs BRCA2 RNA-seq Counts.png",
        width = 800, height = 600, res = 100)

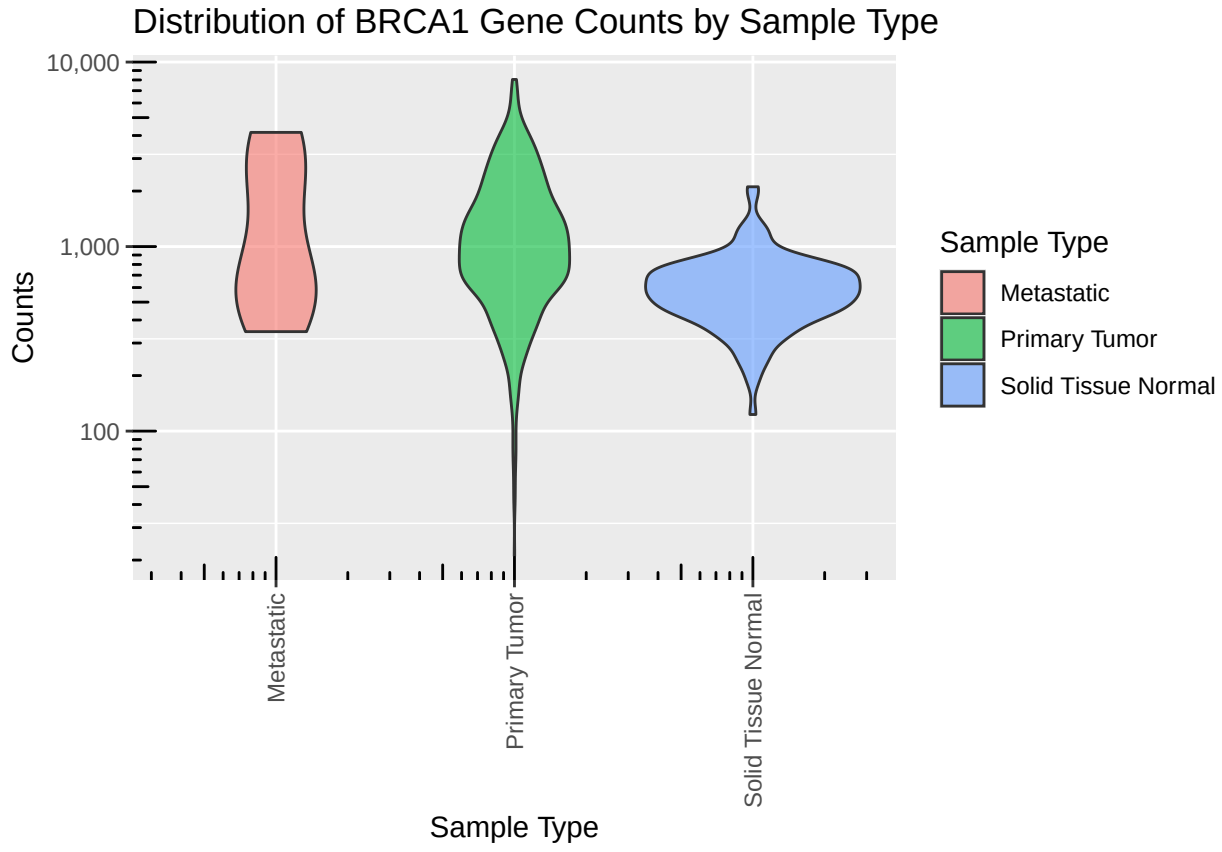
dat.long.genes %>%
  filter(gene_symbols %in% genes.of.interest) %>%
  dplyr::select(-1) %>%
  pivot_wider(names_from = gene_symbols, values_from = Counts) %>%
  filter(!is.na(paper_BRCA_Subtype_PAM50)) %>%
  ggplot(., aes(x = BRCA1,y = BRCA2)) +
  geom_point(aes(colour = factor(paper_BRCA_Subtype_PAM50)),
             alpha = 0.5,
             size = 2,
             na.rm = TRUE) +
             scale_x_log10(labels = scales::comma) +
             scale_y_log10(labels = scales::comma) +
             scale_color_discrete(name = "Factor Values") +  #Legend title
             annotation_logticks(sides = "bl") +
             labs(title = "BRCA1 vs BRCA2 RNA-seq Counts",
                subtitle = "Counts given in logarithmetic scales",
                x = "BRCA1 RNA-sequencing Counts",
                y = "BRCA2 RNA-sequencing Counts"
             ) +
             theme(
                 plot.title = element_text(
                 face = "bold",
                 hjust = 0.5
            ),
                 plot.subtitle = element_text(
                 hjust = 0.5
            )
  )
dev.off()
```

```
## pdf
##   2
```

```r
# c) Create a Violin Plot: Select one covariate from your metadata. Using the count data from
# the first gene and the selected covariate, generate a violin plot that illustrates the
# distribution of count data based on the covariate. For example, if you choose
# "primary_diagnosis", your plot should display a violin plot for each level
# in "primary_diagnosis".

#png("images/Distribution of BRCA1 Gene Counts by Sample_type.png",
#        width = 800, height = 600, res = 100)

dat.long.genes %>%
  filter(gene_symbols == 'BRCA1') %>%
  filter(!is.na(sample_type)) %>%
  ggplot(., aes(x=sample_type,y=Counts, fill = sample_type)) +
  geom_violin(alpha = 0.6) +  #Legend title) +
  labs(
    title = "Distribution of BRCA1 Gene Counts by Sample Type",
    x = "Sample Type",
    y = "Counts",
    fill = "Sample Type"
  ) +
  scale_y_log10(labels = scales::comma) +
  annotation_logticks() +
  theme(axis.text.x = element_text(angle = 90, vjust = 0.5, hjust = 1))
```




```r
#dev.off()
```

```r
count.df <- read.csv(file = "data/counts.csv", header = TRUE,row.names = 1)
meta_data.df <- read.csv(file = "data/meta_data.csv", header = TRUE)
```

```r
# Recommended 10-gene core BRCA/HR variant panel For mutation/variant analysis
genes.of.interest <- c("ENSG00000012048.23","ENSG00000139618.16",
                       "ENSG00000083093.10","ENSG00000138376.11",
                       "ENSG00000108384.15", "ENSG00000185379.20",
                       "ENSG00000136492.9", "ENSG00000149311.19",
                       "ENSG00000183765.23", "ENSG00000167258.15")

gene_name <- c("BRCA1", "BRCA2", "PALB2", "BARD1", "RAD51C",
               "RAD51D", "BRIP1", "ATM", "CHEK2", "CDK12")

count.df.subset <- count.df %>%
  filter(rownames(.) %in% genes.of.interest)

count.df.matrix <- as.matrix(count.df.subset[,])

# Rename from Ensebml ID to Gene
rownames(count.df.matrix) <- gene_name

rownames(count.df.subset) <- gene_name
```